ChatGPT までの距離

足りないのは仕組みか、量か ― そして騙されない側になる

rkskmt

完成した機械を眺める

- 手元にあるもの：82万パラメータ・『吾輩は猫である』1冊で育てた文字レベル GPT
- 世の中にあるもの：ChatGPT

> **クイズ**
>
> 自作GPTと ChatGPT の差は、次のどれが一番大きい？
>
> **a.** 仕組み（アーキテクチャ）が根本的に違う　**b.** 大きさとデータ量　**c.** 訓練後の仕込み

- 答えは **b と c**。仕組みの根っこは、君たちが組んだものと**ほぼ同じ**

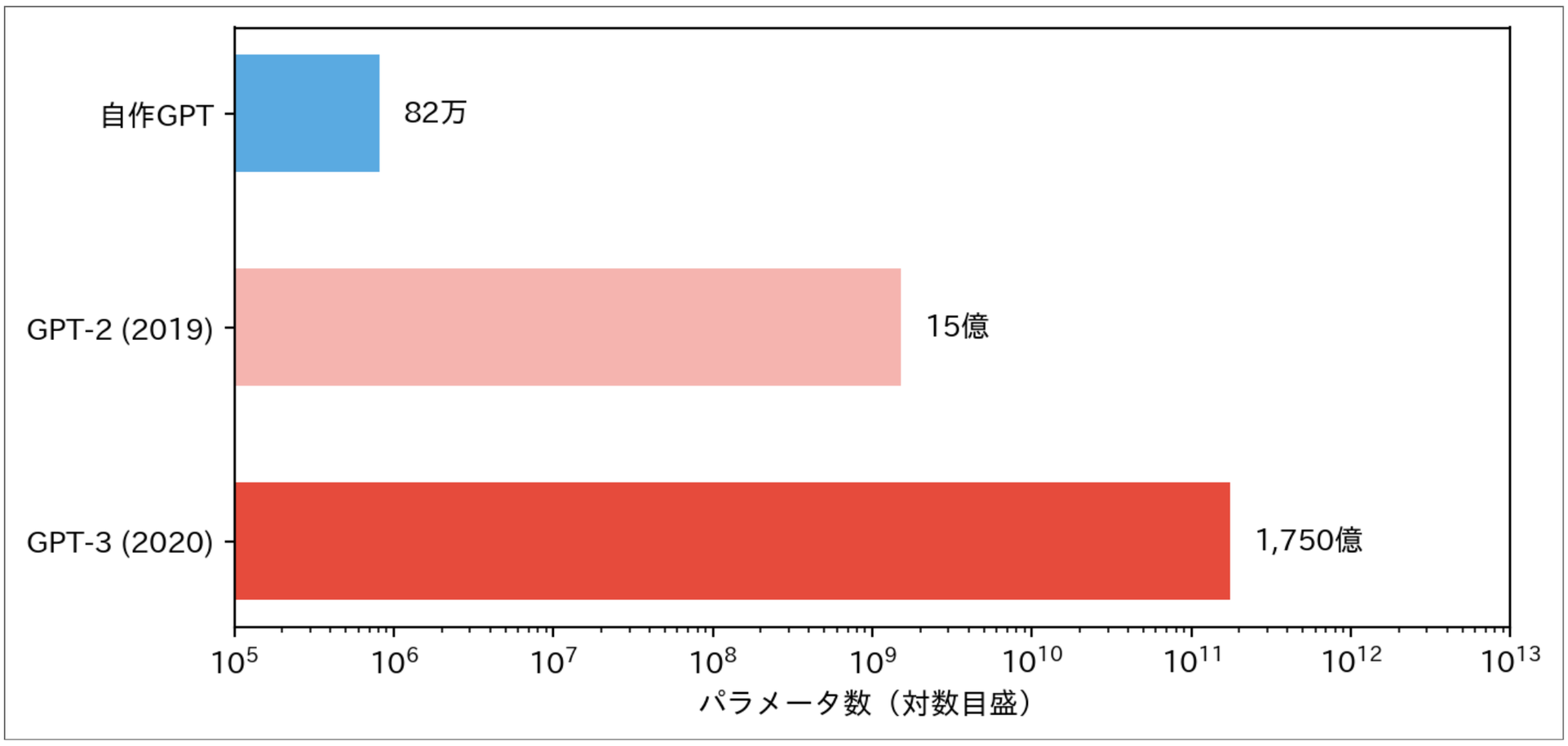
設計図は読めるようになっている

- GPT-2（2019年・OpenAI）の公表スペックを、自作GPTの隣に置く：

	自作GPT	GPT-2	GPT-3
ブロック数（n_layer）	2	48	96
埋め込み次元（n_embd）	96	1,600	12,288
ヘッド数（n_head）	4	25	96
文脈長（block_size）	64	1,024	2,048
パラメータ数	82万	15億	1,750億

- 行の項目名がぜんぶ読める ― **同じ設計図の、数字違い**
- 埋め込み → ブロック×N → 語彙スコア、という背骨は変わらない

距離1：スケール



- GPT-3 は自作GPTの**約21万倍**。データも『猫』32万字に対して**数千億トークン**
- 自作GPTは同じ本を64周して過学習した。巨大 LLM は逆に**データが多すぎて1周も読み切らないことがある** — 暗記する暇がない

大きさを重さで実感する

- パラメータは float32 の数 — **1個が4バイト**。個数を掛ければ「モデルの重さ」が出る

Code

```
for name, p in [("自作GPT", 817_125), ("GPT-2", 1.5e9), ("GPT-3", 175e9)]:
    print(f"{name}\t{p * 4 / 1e9:,.4f} GB")   # 数1個 = 4バイト
```

Result

```
自作GPT   0.0033 GB
GPT-2   6.0000 GB
GPT-3   700.0000 GB
```

- 配布した **ckpt_best.pt** は実測 **3.4MB**（語彙表なども同梱）— メールに添付できる重さ。『猫』本文は約 0.96MB だから、**自作GPTは原作より重い**
- GPT-3 は **700GB** — ノートPCのSSDに丸ごとは入らない。「大きい」はディスクの重さでも実感できる

距離2：語彙 — 文字をやめる

- 組み立ての回の勘定：自作GPTはパラメータの**7割**が語彙まわり（3,045字の出入口）
- 実物は文字でなく**サブワード**を使う：よく出る文字の並びを1トークンに合体させた単位

	自作GPT	GPT-2以降
トークン	文字（3,045種）	サブワード（約5万種）
「こんにちは」	5トークン	1〜2トークン

- 同じ文脈長でも**ずっと長い文章**が窓に入る — では、その約5万種は**誰がどう選んだ**のか？

> **ⓘ ノート**
>
> LLM の料金表が「文字数」でなく「トークン数」なのは、この単位で動いているから。

語彙は数えて作る

- 誰かが5万語を選んだのではない。**よく隣り合う2つを合体**、をひたすら繰り返すだけ — **BPE**（**byte pair encoding**）と呼ぶ
- 『猫』の前半10万字で実際に回した結果：

マージ回数	起きたこと
20回	「って」「る。」に続いて**「主人」「吾輩」「云う」が1トークンに**
100回	最長トークンは「である。」。「吾輩は」「ところ」「そんな」も1トークン

- 数えただけで、『猫』の主役たち（主人・吾輩）が真っ先に単語になる — **語彙はコーパスの鏡**
- トークン数は 100,000 → 91,366（20回）→ 81,286（100回）と縮む — 合体するほど窓に長い文章が入る
- 作り方は回数外の **BPE の付録** で、数え上げから1行ずつ自作する

実物を操作する — ブラウザの中の GPT-2



本物の GPT-2（小型版・1.2億パラメータ）をブラウザ内で動かして中を覗ける公開デモ。ここまでの距離を実物で確かめる。

0. 好きな英文を入れて、右端の**次トークンの確率**を眺める
1. 入力が**サブワード**に割れているのを確認 — 文字でも単語でもない切れ方（距離2の実物）
2. **Temperature** のスライダを動かして、分布が平ら↔尖りに変わるのを確認（生成を操るの回のツマミの実物）
3. 部品の箱を入口から出口までたどって、「習っていない箱」を探す — あってもたいてい名前が違うだけ

> **ⓘ ノート**
>
> この GPT-2 は**事前学習だけ**の素の姿 — 次の距離（仕込み）を、まだ通っていない。

Transformer Explainer を別タブで開く

実験：自作GPTに質問してみる

Code

```python
import torch
from gpt import GPT

torch.set_num_threads(1)
blob = torch.load("data/runs/neko/ckpt_best.pt", weights_only=True)
model = GPT(vocab_size=len(blob["vocab"]), **blob["cfg"])
model.load_state_dict(blob["state_dict"])
model.eval()
chars = blob["vocab"]
stoi = {c: i for i, c in enumerate(chars)}

def gen(prompt, n, seed=0):
    torch.manual_seed(seed)
    idx = torch.tensor([[stoi[c] for c in prompt]])
    out = model.generate(idx, n, temperature=0.8, top_k=40)
    return "".join(chars[i] for i in out[0].tolist())

print(gen("「君の名前は何かね」", 60, seed=1))
```

Result

```
「君の名前は何かね」
「それでね、どうですかも知れないのは、どうです、どうだってもしない」
「あらあ」
「あの伯父さん」
「ええ」と大蔵様は
```

- 返ってきたのは答えではなく、**芝居の続きの台本** — 質問を「登場人物のセリフ」としか読めていない
- 考えてみれば当然：訓練で見た世界には**文の続き**しかない。「質問に答える」という行為は『猫』のどこにも定義されていない

> ⓘ **ノート**
>
> 「質問：〜」「答え：〜」の形式では、そもそも**入力すらできない** — 「：」は 3,045 字の語彙に**ない**（`stoi[":"]` で KeyError）。『猫』に出てこない文字は、自作GPTの世界に存在しない（文字を数にするの回）。

距離3：仕込み — 予測マシンから対話相手へ

- この「続きしか書けない機械」を対話相手に変えるのが3つ目の距離 — 訓練の**あと**の仕込み



- **事前学習**：自作GPTがやったこと、そのもの。ここで「言葉」を覚える
- **指示チューニング**（instruction tuning）：「質問→回答」形式の手本を追加で学習。文の続きではなく**応答**を返すようになる
- **好みの調整**（RLHF — 名前だけ紹介）：人間が「良い答え」を選び、その好みに寄せる
- ChatGPT ＝ **次語予測マシン＋行儀のしつけ**。土台は最後まで次語予測

しつけの中身 — 台本で追加訓練

- 指示チューニングのデータは「指示 → 望ましい応答」の**台本**。人が書いた手本を数万〜数百万組読ませる（形のイメージ）：

指示（人が書く）	応答の手本（人が書く）
日本でいちばん高い山は？	富士山です。標高は 3,776 m です。
「吾輩は猫である」を現代語に言い換えて	私は猫です。

- ここが要点：追加訓練でも**やることは次の1トークン予測のまま**。変わるのは読ませる台本のほう
- 「質問の続き」＝「その答え」という世界で育ち直す — 仕組みには1ミリも手を入れない
- 自作GPTとの違いは訓練の中身ではなく、**コーパスに「答える」という行為が書いてあるかどうか**

実験：自作GPTに事実を聞く

Code

```
print(gen("主人の名前は", 50, seed=1))
```

Result

```
主人の名前はまだそうだね」と迷亭君はますまいと云わない。「え、どうだな。あの娘を貰いた。もっともそんなところで」
```

- 流暢に**何も答えていない**。しかも「名前は**まだ**…」と続けたがる — コーパスで一番有名な「名前」の文脈（「名前はまだ無い」）を引きずっている
- **喋っているのは事実ではなく統計** — 「ありそうな続き」製造機の正直な姿

ハルシネーション — 堂々と間違える理由

- LLM が事実でないことを流暢に述べる現象を**ハルシネーション**（hallucination）と呼ぶ
- 原理は今日見たとおり：モデルの中には「ありそうさ」しかなく、**「本当かどうか」という軸が存在しない**
- スケールが上がると「ありそう」と「本当」の一致率は上がる。だが**原理は消えない** — ChatGPT も、自作GPTの21万倍上手なだけの「続き製造機」

① だから、騙されない側になる

- **検算できるものは検算する**（計算・日付・引用は特に）
- **出典を出させて、実物を確認する**（それらしい架空の出典も平気で作る）
- **流暢さと自信は、正しさの証拠にならない** — 自作GPTがその生き証人

ハルシネーションの解剖

Code

```
print(gen("吾輩の主人は", 60, seed=0))
```

Result

```
吾輩の主人はあるまいと思うくらいならよかろうか。寒月君は猫としてはならんのだから吾輩の頭をあたかも知れない。しかし寒月君において、も
```

- 途中に「**寒月君は猫**としてはならんのだから」— それらしい主張。本文と照合する：

Code

```
text = open("data/neko.txt", encoding="utf-8").read()
print("寒月君は:", text.count("寒月君は"), "回")
print("猫として:", text.count("猫として"), "回")
print("寒月君は猫:", "寒月君は猫" in text)
```

Result

```
寒月君は: 59 回
猫として: 4 回
寒月君は猫: False
```

- 部品はどちらも本文に実在する。**組み合わせだけが作り話**
- 数えるだけの言語モデルの回でやった「でたらめ地名の解剖」と同じ構造 — **一歩一歩は正しいのに、全体は迷子**
- ありそうなかけらを、ありそうな順につないだ結果が、そのまま「堂々とした嘘」になる

演習：AIの発言をファクトチェックする

自作GPTに『吾輩は猫である』の内容を3回語らせ、本文と照合して採点せよ。

0. **neko.txt** ・ **gpt.py** ・ **ckpt_best.pt** を本編と同じ配置に
1. **`gen("吾輩の主人は", 60, seed=n)`** を seed 0〜2 で実行
2. 出力に含まれる「事実らしい主張」に下線を引く（紙でよい）
3. その主張が本文にあるか、**`in`** 演算子や **`count`** で検索して判定（**`"○○" in text`** など）

▸ 解答の型を見る

ここまでの部品でニュースが読める

見かける言葉	正体（習った回）
「コンテキスト長 128K」	**`block_size`**（順序とカンニング）
「トークン課金」	BPE の語彙単位（今日）
「temperature を下げて」	生成のツマミ（前回）
「1.8兆パラメータ」（噂話）	**`sum(p.numel() ...)`**（深く積む）
「事前学習済みモデル」	次語予測の訓練（小さなGPT）

- 仕様表や記事の**専門用語の大半が、自作した部品の名前**になった

まとめ

- ChatGPT との距離は主に：**スケール**（21万倍）・**語彙**（BPE）・**仕込み**（指示チューニング＋好み調整）
- 仕込みも中身は次語予測のまま — 台本を差し替えて「質問の続き＝答え」の世界で育て直すだけ
- 背骨は同じ：埋め込み → Transformer ブロック → 次語予測
- ハルシネーションは故障ではなく原理 — 部品は本物でも、組み合わせに「本当」の保証がない
- 検算する・出典を確かめる・流暢さを信じない — 作った者だけが持てる健全な不信感

現在の設計図　サブワード → 大型Transformer → **指示・好みで追加訓練** → 応答

ⓘ 次回予告：最終回

仕上げは**推し文豪ボット** — 青空文庫から好きな作家を選んで、自分のGPTを育てる。 そして初回の**文豪テスト**の種明かしをする。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：主人の本名を突き止める

自作GPTは「主人の名前」を答えられなかった。本文データから主人（英語教師）の本名を突き止めよ。

1. **neko.txt** を読み込み、`text.count("苦沙弥")` と `text.count("珍野")` を表示
2. 登場回数から、呼び名と姓を推定する（答え合わせは検索エンジンでどうぞ）

問題2：スケールの倍率

自作GPT（817,125）に対する GPT-2（15億）・GPT-3（1,750億）の倍率を計算せよ。GPT-3の倍率を「自作GPT〇〇台ぶん」と読み上げられる形で表示すること。

解答例

▶ 解答例を見る

Result

```
苦沙弥: 99 回
珍野: 2 回
GPT-2: 自作GPT 1,836 台ぶん
GPT-3: 自作GPT 214,166 台ぶん
```